

Understanding Software Defect Prediction: A Large-scale Empirical Study Across Uncertainty Quantification and Performance Evaluation

RANJUN PENG, School of Computer Science and Engineering, Macau University of Science and Technology, China

XUAN XIE*, School of Computer Science and Engineering, Macau University of Science and Technology, China

RUBING HUANG, School of Computer Science and Engineering, Macau University of Science and Technology, China

WENBIN HE, State Key Laboratory of Intelligent Manufacturing Equipment and Technology, School of Mechanical Science and Engineering, Huazhong University of Science and Technology, China

ZHIJIE WANG, Department of Computer Science and Software Engineering, Concordia University, Canada

Software defect prediction (SDP) classifiers produce probabilities used for inspection prioritization, threshold tuning, and risk communication. Probability-based uncertainty quantification (UQ) characterizes prediction confidence, but whether common UQ metrics reliably indicate performance and calibration remains unclear. We conducted a large-scale empirical study of probability-based UQ for SDP. We evaluated five UQ metrics, six performance metrics, and three calibration metrics for 16 representative classifiers. We analyzed these relationships under two prediction settings: within-project defect prediction (WPDP), using 36 benchmark datasets, and cross-project defect prediction (CPDP), using 32 feature-compatible datasets. Results showed that UQ was highly context-dependent. Under WPDP, UQ correlated more consistently with false positive rate and AUC than with MCC, F1 score, and other metrics; these correlations also varied across classifier categories and dataset collections. Performance and calibration were related but not interchangeable; classifiers with strong discrimination could still exhibit large calibration error. Under CPDP, several UQ-performance and UQ-calibration correlations weakened or reversed, indicating that uncertainty signals do not reliably transfer across projects. Thus, UQ should be evaluated against specific performance objectives. Calibration should be assessed independently using multiple metrics. Transferred probabilities should be revalidated before guiding quality-assurance decisions.

CCS Concepts: • **Software and its engineering** → **Software defect prediction**; • **Computing methodologies** → *Supervised learning by classification; Ensemble methods.*

*Corresponding author.

Authors' Contact Information: **Ranjun Peng**, 1230024861@student.must.edu.mo, School of Computer Science and Engineering, Macau University of Science and Technology, Macao SAR, China; **Xuan Xie**, xiexuan@must.edu.mo, School of Computer Science and Engineering, Macau University of Science and Technology, Macao SAR, China; **Rubing Huang**, rbhuang@must.edu.mo, School of Computer Science and Engineering, Macau University of Science and Technology, Macao SAR, China; **Wenbin He**, bobo_hewenbin@163.com, State Key Laboratory of Intelligent Manufacturing Equipment and Technology, School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, China; **Zhijie Wang**, zhijie.wang@concordia.ca, Department of Computer Science and Software Engineering, Concordia University, Montreal, Quebec, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/6-ART

<https://doi.org/>

Additional Key Words and Phrases: Software defect prediction, uncertainty quantification, predictive performance, calibration, cross-project defect prediction

ACM Reference Format:

RanJun Peng, Xuan Xie, Rubing Huang, Wenbin He, and Zhijie Wang. 2026. Understanding Software Defect Prediction: A Large-scale Empirical Study Across Uncertainty Quantification and Performance Evaluation . 1, 1 (June 2026), 30 pages.

1 Introduction

Software defect prediction (SDP) aims to identify defect-prone modules before release, so that quality-assurance effort can be focused where it is most likely to improve software quality. This task has been studied for more than two decades, from early work on static code attributes to large classifier benchmarks and systematic reviews [8, 23, 37, 40, 44–46]. Recent work has expanded SDP through deep representations, attention-based feature fusion, and cross-project learning [41, 50, 54, 58, 79]. Nevertheless, metric-based tabular SDP remains the most popular, as a significant number of public benchmarks and industrial pipelines still describe modules using product, process, and change metrics [25, 48, 49, 82].

SDP is commonly formulated as a binary classification task, where a model predicts whether a software module is defective or clean. To make this prediction, learning-based SDP models typically estimate the probability that each module contains defects and then classify modules based on a decision threshold. A higher predicted probability indicates a greater likelihood that the module is defective. Beyond binary classification, these probabilities are often used for other practical purposes, such as prioritizing modules for inspection, adjusting inspection thresholds to match available resources [7], combining predictions from multiple models, and communicating defect risk to developers [80]. However, predicted probabilities alone may not fully capture the reliability of a prediction. Two modules may receive similar defect probabilities while differing substantially in the confidence that the model has in those estimates. To address this limitation, uncertainty quantification (UQ) has been introduced to characterize the ambiguity associated with model predictions [12]. By providing an additional measure of confidence, UQ has the potential to support more informed decision-making in SDP.

Over the last decade, the machine learning community has developed a series of UQ techniques, including probability-based measures, Bayesian approximations, and ensemble-based approaches [1, 16, 27–29, 35, 55]. Among the various UQ approaches, probability-based uncertainty measures are particularly attractive because they can be derived directly from the predicted class probabilities produced by most SDP models, requiring no changes to the underlying learning algorithm. Common examples include confidence, margin, and entropy-based measures, all of which aim to quantify how certain a model is about its prediction. These measures have also recently been adopted in software engineering tasks such as test selection and automated program repair [15, 43, 73, 77, 78]. More recently, defect prediction researchers have investigated techniques such as conformal prediction and probability calibration in just-in-time defect prediction settings [30, 61, 62]. However, despite the growing interest in uncertainty-aware defect prediction, there remains limited empirical evidence regarding *which probability-based uncertainty measures are most effective for SDP*. Furthermore, practitioners lack clear guidance on *how uncertainty scores should be interpreted and incorporated into defect prediction workflows*.

Answering these questions is challenging because SDP classifiers generate probability estimates through fundamentally different mechanisms; for example, probabilities may be derived from likelihood assumptions, sigmoid-transformed margins, neighborhood counts, tree leaf frequencies, softmax activations, or ensemble aggregation. As a result, the same uncertainty measure may behave differently across classifiers and data distributions. This challenge is particularly pronounced in

SDP, where datasets often exhibit substantial class imbalance, class overlap, label noise, and project-specific feature characteristics [2, 20, 71]. The challenge is further compounded by differences in feature distributions, defect rates, and labeling practices in cross-project defect prediction (CPDP) [51, 55, 83]. It remains unclear whether probability-based uncertainty measures provide consistent and reliable signals across different defect prediction settings.

To bridge the gap, we conducted a large-scale empirical study to examine whether probability-based UQ can be reliably used as empirical evidence for SDP. Specifically, we investigated the following research questions.

- RQ1 Can probability-based UQ serve as a general performance indicator in within-project defect prediction?
- RQ2 Does high predictive performance imply well-calibrated probabilities in within-project defect prediction?
- RQ3 Do relationships among UQ, performance, and calibration persist under cross-project defect prediction?

To answer these questions, we collected 40 candidate SDP classifiers and selected 16 behaviorally representative classifiers using classifier output difference clustering [5, 34]. Our evaluation spanned 36 datasets from AEEEM, NASA, PROMISE, and ReLink [10, 31, 64, 74]. For each classifier–dataset pair, we computed five probability-based UQ metrics, six predictive performance metrics, and three calibration metrics. We then investigated the relationships among uncertainty, predictive performance, and calibration under two prediction settings: within-project defect prediction (WPDP), covering all 36 datasets, and cross-project defect prediction (CPDP), covering 32 feature-compatible datasets.

The study yielded three key observations that challenge common assumptions in uncertainty-aware SDP evaluation. The first two concern WPDP. First, probability-based uncertainty metrics were informative for only a narrow subset of performance measures, most notably false positive rate (FPR) and AUC; for commonly reported metrics such as F1 score, the observed relationships were weak or negligible. Second, predictive performance and probability calibration were related but not interchangeable, as classifiers with strong predictive performance could still produce poorly calibrated probability estimates, limiting the reliability of threshold-based decision making. The third observation concerns the transfer from WPDP to CPDP: relationships between uncertainty and predictive quality were not stable across the two settings, and several relationships observed under WPDP weakened or even reversed under CPDP, suggesting that uncertainty evidence obtained within a project cannot be assumed to generalize to other projects.

In summary, this paper makes the following contributions:

- We conducted a large-scale empirical study of probability-based uncertainty quantification (UQ) for software defect prediction (SDP), covering 16 representative classifiers and 36 benchmark datasets under both WPDP and CPDP settings.
- We systematically characterized the relationships between probability-based uncertainty metrics, predictive performance, and probability calibration across classifier categories, dataset collections, and prediction granularities.
- We identified the strengths, limitations, and practical applicability of probability-based UQ methods in SDP, providing evidence-based guidance for researchers and practitioners on selecting and interpreting uncertainty metrics.
- Our replication package is publicly available at <https://anonymous.4open.science/r/TOSEM26-SDP-UQ-E8B6>.

The remainder of the paper is organized as follows. Section 2 reviews defect prediction, dataset and transfer challenges, and probabilistic predictive evaluation. Section 3.2 describes the study

design and protocol. Sections 3.3–3.5 report the results for the three research questions. Section 4 discusses the implications of our findings for uncertainty-aware SDP evaluation and reporting.

2 Related Work

This study drew on three lines of work: defect prediction classifiers, dataset and transfer challenges, and probabilistic predictive evaluation. Prior SDP research has examined benchmark classifiers, transfer learning, class imbalance, and project-specific data distributions. UQ research has proposed measures of predictive uncertainty, but most evidence comes from general machine learning and deep learning. Prior work has shown that defect prediction performance is sensitive to classifier choice and dataset differences, and that probability estimates require evaluation separately from classification performance. What remains unresolved is whether probability-based UQ metrics from heterogeneous classical classifiers provide consistent performance indicators in metric-based module-level SDP. This section reviews the evidence behind that gap.

2.1 Software Defect Prediction

Software defect prediction aims to identify defect-prone modules before release so that inspection and testing effort can be allocated to modules with higher defect risk [82]. Many SDP studies formulate the task as binary classification using product metrics, process metrics, or both. Common classifiers in these studies include Logistic Regression, Naive Bayes, Support Vector Machines, Decision Trees, and ensemble learners [36, 37, 44, 68]. Process metrics such as code churn and developer activity also improve defect prediction [48, 49]. When a project has little labeled history, cross-project defect prediction transfers training data from other projects [50, 51, 83]. Work on semantic code representations has expanded, but metric-based tabular SDP remains relevant because many public and industrial datasets use software metrics [54, 58, 79, 82].

Benchmark studies have shown that classifier evaluation in SDP is sensitive to validation protocol, parameter tuning, and performance metric choice [17, 37, 67, 69]. This matters for UQ because uncertainty scores are derived from the same probability outputs produced by these classifiers. These studies have shown that classifier choice and validation design shape reported performance. They have left open whether probability outputs from these classifiers yield uncertainty scores that track performance or calibration.

2.2 Dataset and Transfer Challenges in Software Defect Prediction

Defect data contain class imbalance, class overlap, label noise [33], and project-specific feature distributions [14, 20, 64, 71]. These properties affect both classification performance and probability estimates. Class imbalance can bias the learned decision boundary toward the clean class, while overlap and noise make some modules difficult to classify [42, 65].

SDP studies have used class imbalance treatments [21, 69, 72]. Random undersampling, SMOTE, Borderline SMOTE, and ADASYN change the training distribution to improve minority class detection [13, 24, 26]. However, imbalance treatment does not remove all sources of dataset heterogeneity. Prior work has shown that overlap and noisy labels can remain relevant after imbalance correction [2, 20, 71]. Oversampling can introduce synthetic points in ambiguous regions, which may affect probability estimates [2, 4]. These data challenges motivate an evaluation of whether probability-based uncertainty behaves consistently across dataset collections, prediction granularities, and cross-project transfer settings.

2.3 Uncertainty Quantification

Uncertainty quantification characterizes ambiguity in classifier outputs [1, 27, 29]. In this paper, UQ refers to uncertainty metrics derived from predictive probabilities. In software engineering,

uncertainty-aware methods support test selection and automated program repair, where entropy or confidence scores can help prioritize inputs or candidate patches [43, 78].

Calibration is separate from predictive uncertainty. A classifier is calibrated when predicted probabilities match empirical outcome frequencies [22, 52, 53]. Calibration has been studied for defect prediction and imbalanced classification settings [9, 70]. However, high classification performance does not ensure calibrated probabilities. This is relevant for SDP because predicted probabilities may support threshold setting, inspection prioritization, and cost-sensitive decisions.

Recent work has examined uncertainty-aware prediction in just-in-time defect prediction. Shahini et al. studied conformal prediction for just-in-time defect prediction and evaluated whether conformal prediction can identify predictions that are likely to be incorrect [62]. Related work has also examined calibration in just-in-time defect prediction and shown that probability-based decision support requires explicit calibration assessment [61]. These studies have addressed predictive confidence in change-level just-in-time settings. They have not answered whether probability-based uncertainty from heterogeneous classical classifiers serves as a general signal for performance, calibration, or cross-project transfer in module-level SDP.

Three empirical gaps remain. First, prior work has not established whether probability-based UQ metrics correlate with predictive performance across the heterogeneous classifiers and data conditions in SDP. Second, it has not been shown whether high predictive performance implies well-calibrated probabilities in SDP. Third, it is unknown whether UQ–performance and UQ–calibration correlations persist under cross-project transfer. We addressed these gaps by evaluating correlations between UQ and performance and between UQ and calibration across classifiers, dataset collections, and prediction settings.

3 Empirical Study of Uncertainty, Predictive Performance, and Calibration

Building on the three knowledge gaps identified in Section 2.3, this empirical study treated probability-based UQ as a measurable signal rather than as an assumed indicator of model quality. The study examined whether uncertainty scores derived from classifier probability outputs aligned with predictive performance, calibration, and transfer behavior in SDP. We used a common evaluation protocol across classifiers and datasets, which allowed UQ behavior to be compared across classifier categories, dataset collections, prediction granularities, and prediction settings. In this study, uncertainty denotes ambiguity in the classifier probability distribution for an individual software module. A prediction close to an even split, such as 0.51 versus 0.49, is more uncertain than a concentrated prediction.

3.1 Study Motivation and Research Questions

Three knowledge gaps motivate the research questions. Each gap identifies a specific aspect of probability-based UQ in SDP that lacks systematic empirical evidence.

Gap 1: It remains unclear whether probability-based UQ correlates with predictive performance across heterogeneous classifiers and datasets. Classical SDP benchmarks employ a wider set of classifiers than typical UQ studies, including Logistic Regression, Decision Tree, Naive Bayes, Support Vector Machine, Random Forest, and KNN [37, 68]. These classifiers estimate probabilities through fundamentally different mechanisms, such as likelihood assumptions, link functions, leaf frequencies, neighborhood counts, margin transformations, or ensemble aggregation [52]. SDP datasets further vary in class imbalance [48, 72] and prediction granularity (file-, class-, and method-level). Prior UQ research has focused predominantly on deep learning [1, 16, 35], and it has not established whether probability-based UQ provides a consistent performance signal across the diverse classifiers and data conditions found in SDP. This gap motivates **RQ1**.

Gap 2: High predictive performance has not been shown to imply well-calibrated probabilities in SDP. Predictive performance and probability calibration measure different properties of classifier outputs. A classifier can separate defective modules effectively while assigning probability values that do not match empirical defect frequencies [22, 52]. Conversely, a classifier may exhibit low aggregate calibration error while providing weak discrimination for a target metric such as MCC or AUC. These properties support different decisions: discrimination guides inspection ranking, whereas calibration matters when probability values are interpreted as defect likelihoods. Although prior work has studied calibration in general machine learning [22, 53] and just-in-time defect prediction [61], systematic evidence on whether high-performing SDP classifiers also produce well-calibrated probabilities remains lacking. This gap motivates **RQ2**.

Gap 3: It is unknown whether UQ-performance and UQ-calibration correlations observed within a project persist under cross-project transfer. CPDP introduces distribution shifts in feature distributions, defect rates, and labeling practices between source and target projects [51, 55, 83]. Class imbalance treatments such as SMOTE further alter the training distribution and may affect probability estimates [9, 70]. These shifts can change both the classifier probability outputs and the data conditions under which UQ metrics are computed. However, no prior study has examined whether the UQ correlations established in within-project settings remain stable after cross-project transfer. This gap motivates **RQ3**.

These three gaps together call for a systematic evaluation of probability-based UQ in SDP. Accordingly, the study is organized around three research questions:

- **RQ1:** Can probability-based UQ serve as a general performance indicator in within-project defect prediction?
- **RQ2:** Does high predictive performance imply well-calibrated probabilities in within-project defect prediction?
- **RQ3:** Do relationships among UQ, performance, and calibration persist under cross-project defect prediction?

3.2 Study Design

3.2.1 Benchmark Datasets. The benchmark contains 36 defect prediction datasets from four established repositories: AEEEM [10], NASA [64], PROMISE [31], and ReLink [74]. We adopted the same curated dataset suite as Wan et al. [71]. The benchmark comprises 5 AEEEM datasets, 12 NASA datasets, 15 PROMISE datasets, and 4 ReLink datasets. These corpora are common in defect prediction research [39, 67, 69]. They span different prediction granularities: AEEEM and PROMISE are treated as class-level datasets, NASA as function/method-level datasets, and ReLink as file-level datasets. Prediction granularity refers to the structural level of the software artifacts being analyzed, such as files, classes, or individual functions. Because granularity is coupled with dataset collection in this benchmark, granularity-level analyses are interpreted as granularity-aware collection comparisons rather than as evidence of a pure granularity effect.

3.2.2 Candidate Classifiers. We constructed the candidate classifier pool from 40 classifiers. Following the taxonomy in prior studies [71], we organized these candidates into the following groups:

- **Probabilistic Model:** This group includes *Naive Bayes*, *Complement Naive Bayes*, *Gaussian Process Classifier*, and *Quadratic Discriminant Analysis*.
- **Linear Model:** This group includes *Ridge Classifier*, *Logistic Regression*, *Linear Support Vector Classifier*, *Stochastic Gradient Descent Classifier*, and *Passive Aggressive Classifier*.
- **Instance-Based and Kernel Method:** This group includes *KNN*, *Linear Support Vector Machine*, and *Radial Basis Function Kernel Support Vector Machine*.

- **Tree- and Rule-Based Method:** This group includes *Classification and Regression Tree*, *Boosted Rule Set*, and *Greedy Rule List*.
- **Neural Network:** This group includes two multilayer perceptrons: *One Hidden Layer Multilayer Perceptron* and *Two Hidden Layer Multilayer Perceptron*.
- **General Ensemble Method:** This group includes *Random Forest*, *Extremely Randomized Trees*, *Gradient Boosted Decision Trees*, *Voting Classifier*, *Extreme Gradient Boosting*, *Light Gradient Boosting Machine*, *Categorical Boosting*, *Histogram-Based Gradient Boosting Classifier*, and *Adaptive Boosting Classifier*.
- **Bagging and Boosting Wrapper:** This group includes *Naive Bayes Bagging*, *Logistic Regression Bagging*, *Support Vector Machine Bagging*, *Decision Tree Bagging*, *Multilayer Perceptron Bagging*, *Naive Bayes Boosting*, *Logistic Regression Boosting*, *Support Vector Machine Boosting*, *Decision Tree Boosting*, and *Multilayer Perceptron Boosting*.
- **Imbalance-Aware Ensemble:** This group includes *Balanced Random Forest Classifier*, *Easy Ensemble Classifier*, *Random Under Sampling Boosting Classifier*, and *Balanced Bagging Classifier*.

To reduce redundancy over the candidate classifiers, we used *Classifier Output Difference (COD)*, which measures behavioral distance by prediction disagreement [5, 34]. For two classifiers f_a and f_b on the same test set of n instances, COD is the proportion of instances on which their predicted labels differ: $\text{COD}(f_a, f_b) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[f_a(x_i) \neq f_b(x_i)]$. A COD score of 0 indicates identical predictions on all test instances, whereas a score approaching 1 indicates disagreement on nearly every prediction.

To compute COD distances that reflect the actual SDP behavior, we followed the methodology of Wan et al. [71]. We evaluated each candidate classifier on the 36 benchmark datasets using 5×5 repeated cross-validation. For each sample in every dataset, we obtained the majority-vote prediction across all CV folds from each classifier. We then computed pairwise COD values per dataset and averaged the resulting COD matrices across the 36 datasets to produce a single 40×40 distance matrix. This procedure ensured that the behavioral distances captured classifier disagreement patterns across the benchmark defect prediction tasks.

Using this pairwise COD distance matrix, we applied agglomerative hierarchical clustering with average linkage to group the 40 classifiers. The clustering result is visualized as a dendrogram (Figure 1). Each leaf node represents one candidate classifier. The vertical axis shows the COD distance at which two branches merge; shorter merge distances indicate higher behavioral similarity. We chose the COD cut point according to two criteria: preserving behavioral diversity among classifiers and avoiding over-representation of behaviorally redundant variants. Specifically, we inspected the hierarchical COD dendrogram and selected the cut before a large increase in merge distance, where further merging would combine classifiers with visibly different prediction behaviors. This cut produced 16 clusters. The resulting clusters retained the major probability-generation mechanisms in the candidate pool, including probabilistic models, margin-based models, tree/rule-based models, neural networks, general ensembles, and imbalance-aware ensembles, while collapsing many wrapper variants that exhibited similar prediction behavior. Thus, the number 16 was not chosen as a target model count, but resulted from a COD-based redundancy reduction criterion fixed before the RQ analyses. Within each cluster, we selected one representative classifier using the following priority rules: preserving underrepresented learning categories, preferring base learners over wrappers when applicable, selecting the cluster medoid, and finally using the predefined candidate order to break ties. As a result, we retained 16 representative classifiers from the original pool of 40 candidates. Table 1 summarizes these representative classifiers.

Probabilistic classifiers estimate class likelihoods or decision boundaries from distributional assumptions. Linear classifiers produce scores via a linear decision boundary transformed by a

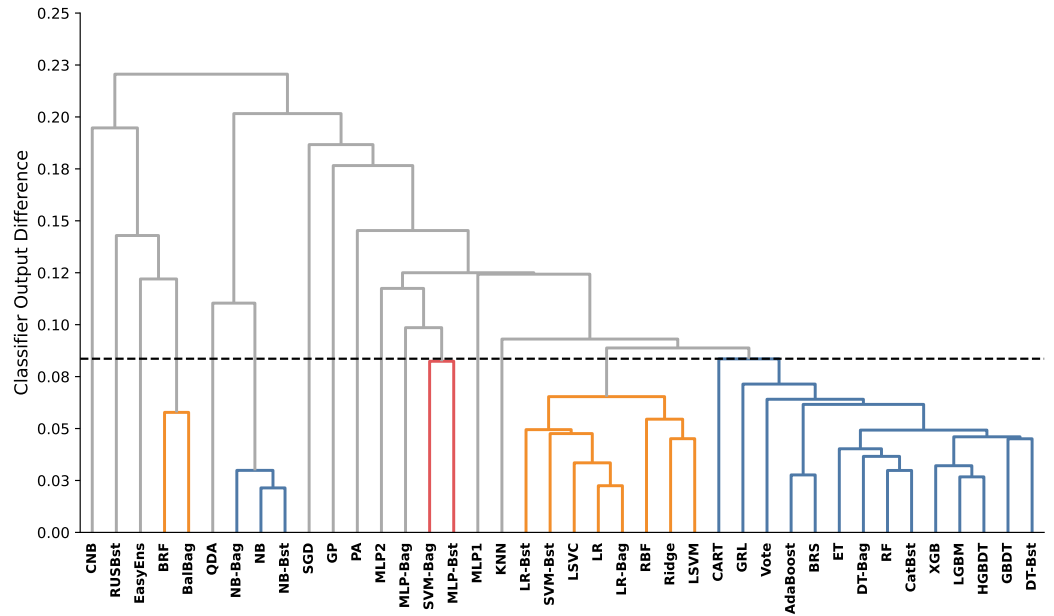


Fig. 1. Dendrogram of 40 candidate classifiers clustered using COD scores. Each leaf node at the bottom represents one candidate classifier. The vertical axis indicates COD distance: classifiers connected at shorter distances exhibit more similar prediction behavior. The dashed horizontal line marks the threshold used to partition the dendrogram. Each connected subtree below this threshold represents one cluster, and the leaves within the same subtree belong to the same cluster. This threshold yields 16 clusters, and one representative classifier remains in each cluster, resulting in the final 16 classifiers.

Table 1. Categorization of the 16 representative machine learning classifiers obtained by COD clustering.

Category	Selected Representative Classifiers
Probabilistic Model	Naive Bayes, Complement Naive Bayes, Gaussian Process Classifier, Quadratic Discriminant Analysis
Linear Model	Logistic Regression, Stochastic Gradient Descent Classifier, Passive Aggressive Classifier
Lazy Learning	KNN
Decision Tree	Classification and Regression Tree
Neural Network	One Hidden Layer Multilayer Perceptron, Two Hidden Layer Multilayer Perceptron, Multilayer Perceptron Bagging
SVM-Based	Support Vector Machine Bagging
Ensemble	Balanced Random Forest Classifier, Easy Ensemble Classifier, Random Under Sampling Boosting Classifier

sigmoid link. Lazy learning derives probabilities from local neighborhood density; KNN is the sole representative because it is the only lazy learner in the candidate pool and its distance-based probability mechanism is distinct from all other categories. Decision trees estimate probabilities from leaf node class frequencies. Neural Network classifiers output softmax probabilities from nonlinear hidden representations. MLP Bagging is placed in this category because its base learner is an MLP that produces probabilities through the softmax mechanism. The bagging wrapper reduces variance through bootstrap aggregation [6] but does not alter the underlying probability generation process. SVM-Based classifiers rely on margin transformations to convert signed margins into probabilities, a mechanism distinct from softmax, leaf frequency estimates, and likelihood-based estimates. The Ensemble category uses class-balancing mechanisms that alter the effective class prior seen by the base learner, making its probability behavior different from that of the other six categories.

3.2.3 Metrics.

- **Uncertainty Metrics [76]:** Uncertainty reflects the confidence in a classifier's output. We used five common uncertainty metrics (Maximum Probability [28], Least Confidence [38], Margin Score [59], Predictive Entropy [63], and DeepGini [15]), as summarized in Table 2.
- **Performance Metrics:** The study included MCC, Precision, Recall, FPR, AUC, and F1 score, which are widely adopted in SDP research [2, 40, 56].
- **Calibration Metrics:** Calibration measures the degree to which a model's prediction accurately reflects the true likelihood of an event. We used Negative Log Likelihood (NLL), Brier Score, and Expected Calibration Error (ECE) [18, 19, 47].
- **Correlation Metrics:** We used three correlation coefficients: Pearson's r [11, 60], Spearman's ρ [66], and Kendall's τ [32].

Table 3 provides descriptions for all performance, calibration, and correlation metrics.

Table 2. Uncertainty metrics in the study.

Metric	Key Formulation	Description
Maximum Probability	$MP(x) = \max_c p(c x)$	This metric uses the highest predicted class probability as a certainty score. A lower value indicates higher uncertainty.
Least Confidence	$LC(x) = 1 - \max_c p(c x)$	This metric takes the complement of the most likely class probability. A higher value indicates higher uncertainty.
Margin Score	$M(x) = p(\hat{y}_1 x) - p(\hat{y}_2 x)$	This metric computes the difference between the largest and second-largest predicted probabilities. A lower value indicates higher uncertainty.
Predictive Entropy	$H(y x) = -\sum_{c=1}^C p(y_c x) \log p(y_c x)$	This metric measures the dispersion of the predicted class distribution. A higher value indicates higher uncertainty.
DeepGini	$DG(x) = 1 - \sum_{c=1}^C p(y_c x)^2$	This metric is derived from the Gini impurity index. A higher value indicates a less concentrated predictive distribution and therefore higher uncertainty.

Table 3. Performance, calibration, and correlation metrics in the study.

Metric	Formulation	Description
PERFORMANCE METRICS		
MCC	$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$	The Matthews Correlation Coefficient is a balanced measure that is less sensitive to class imbalance, ranging from -1 to +1.
Precision	$P = \frac{TP}{TP+FP}$	This metric reports the ratio of correctly predicted positives to all predicted positives.
Recall	$R = \frac{TP}{TP+FN}$	This metric, also known as Probability of Detection, reports the ratio of correctly predicted positives to all actual positives.
F1 score	$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$	The F1 score is the harmonic mean of Precision and Recall, balancing both false positives and false negatives.
FPR	$FPR = \frac{FP}{FP+TN}$	This metric measures the ratio of actual negatives incorrectly classified as positive. A larger FPR indicates more false alarms during inspection.
AUC	$\int_0^1 TPR(t) d(FPR(t))$	The Area Under the ROC Curve (AUC) evaluates the classifier's ability to discriminate between classes across all thresholds.
CALIBRATION METRICS		
ECE	$\sum_{m=1}^M \frac{ B_m }{n} \bar{y}(B_m) - \bar{p}(B_m) $	The Expected Calibration Error measures the gap between the empirical positive rate and the mean positive-class probability across M bins.
NLL	$NLL = -\frac{1}{n} \sum_{i=1}^n \log p(y_i x_i)$	The Negative Log Likelihood evaluates probabilistic prediction error. Lower values indicate better probabilistic predictions.
Brier Score	$BS = \frac{1}{n} \sum_{i=1}^n (p(y=1 x_i) - y_i)^2$	The Brier Score measures the mean squared error between predicted probabilities and observed outcomes. Lower values indicate better probabilistic predictions.
CORRELATION COEFFICIENTS		
Spearman's ρ	$1 - \frac{6 \sum d_i^2}{n(n^2-1)}$	This nonparametric coefficient measures monotonic correspondence between uncertainty and performance.
Kendall's τ	$\frac{C-D}{\sqrt{(C+D+T_u)(C+D+T_v)}}$	This coefficient measures ordinal correlation via concordant/discordant pairs and is less sensitive than Spearman's ρ to extreme values.
Pearson's r	$\frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$	This coefficient measures the linear correlation between variables and serves as a baseline for linear dependency.

3.2.4 Experimental Protocol. We repeated all experiments with 30 independent random seeds and reported the average over these runs. For classifiers that returned only a decision function, we transformed the raw score s into a score vector with two entries, $[1 - \sigma(s), \sigma(s)]$, where $\sigma(s) = 1/(1 + \exp(-s))$. This choice is motivated by Platt's sigmoid calibration method for SVM decision values, which uses a sigmoid function to map margin scores to probability estimates [52, 57, 75, 81]. Here, a decision function denotes an uncalibrated score, such as a signed margin or distance from the decision boundary, rather than a probability. We did not assume that either the directly reported probabilities or the sigmoid-transformed scores were calibrated; our calibration analysis evaluated probability calibration separately.

For each classifier and dataset, we recorded three groups of quantities. First, we computed the six predictive performance metrics listed in Table 3. Second, we computed the three calibration metrics from the predicted probabilities. Third, we computed the five UQ metrics. After seed averaging, each WPDP classifier and dataset pair contributed one observation containing performance, calibration, and UQ values.

For RQ1, we correlated each UQ metric with each performance metric using Pearson's r , Spearman's ρ , and Kendall's τ . The global analysis used all classifier and dataset observations. Analyses by dataset collection computed the same correlations separately for AEEEM, NASA, PROMISE, and ReLink. Granularity analyses followed the dataset mapping described in Section 3.2: AEEEM and PROMISE are class-level datasets, NASA is function- or method-level, and ReLink is file-level. For RQ2, we computed Spearman correlations between the performance metrics and the calibration metrics across the same WPDP observations.

For RQ3, we evaluated CPDP without transfer learning [51]. Within each included dataset collection, one dataset served as the source project and a different dataset from the same dataset collection served as the target project. We treated source-to-target pairs as ordered, so we evaluated both $A \rightarrow B$ and $B \rightarrow A$ for two datasets A and B . For each ordered pair, we trained each of the 16 classifiers on the source project and evaluated it directly on the target project. We aligned source and target features by common feature names, fitted scaling parameters only on the source project, and then applied them to the target project. The CPDP analysis included AEEEM, NASA, and PROMISE, covering 32 datasets. We excluded ReLink from CPDP because its four datasets formed two feature-incompatible subgroups. Because these two subgroups shared no common features, ReLink did not provide enough source-to-target pairs within the same dataset collection for a stable CPDP analysis.

3.3 RQ1: Can probability-based UQ serve as a general performance indicator in within-project defect prediction?

RQ1 and RQ2 are conducted under WPDP, where the training and test data come from the same project; RQ3 then revisits these relationships under CPDP. To address RQ1, we examined whether uncertainty estimates were correlated with predictive performance under WPDP and how this correlation varied across classifier categories, granularities, and datasets. We organized the evaluation around the following three dimensions.

- **RQ1.1 (Global Correlation).** We first computed global correlations between UQ and performance across all classifier and dataset pairs.
- **RQ1.2 (Category-Level Alignment).** We assessed whether alignment between UQ and performance differed across classifier categories.
- **RQ1.3 (Granularity-Level Correlation).** We examined whether correlations between UQ and performance differed across dataset collections (AEEEM, NASA, PROMISE, ReLink) and across prediction granularities (class-level, function/method-level, file-level).

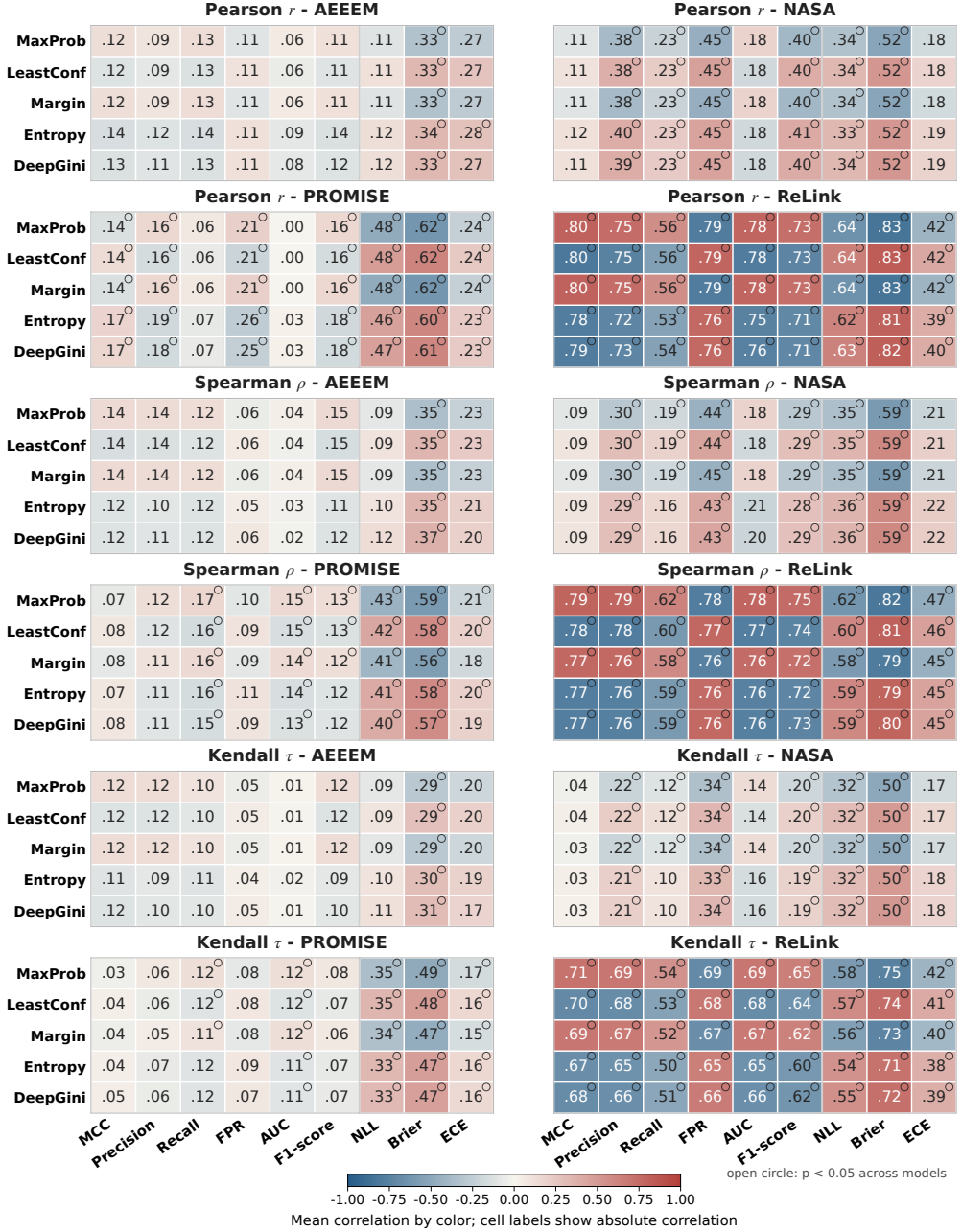


Fig. 2. **Correlations between UQ and performance by dataset collection and correlation method.** Each heatmap shows the mean correlation across the classifiers between five UQ metrics (rows) and nine target metrics (columns). Color scale: blue (−1) to red (+1). Cell labels report absolute correlation magnitudes on the original correlation scale; the sign is encoded by color. Open circles mark cells where the 16 classifier-level correlations pass a one-sample t test at $p < 0.05$, indicating that the mean correlation across classifiers is statistically distinguishable from zero. ReLink shows larger correlations than the other three collections.

3.3.1 RQ1.1: Global Correlation Between UQ and Performance. The experiment used 36 datasets and 16 classifiers. We computed Pearson's r , Spearman's ρ , and Kendall's τ between the five UQ metrics and six performance metrics. Figure 2 shows the correlation matrices for Pearson r , Spearman ρ , and Kendall τ across each dataset collection. The heatmap color scale represents the average correlation magnitude and polarity across the 16 classifiers. Cell labels report absolute correlation magnitudes on the original correlation scale, with the sign encoded by color. Open circles denote cells where the classifier-level correlations achieve statistical significance via a one-sample t test ($p < 0.05$). Aggregated across all dataset collections, the majority of correlations between UQ and performance remained marginal.

The most prominent signal involved the false positive rate (FPR). Maximum Probability had a negative correlation with FPR (Spearman $\rho \approx -0.48$; Kendall $\tau \approx -0.38$), whereas uncertainty-oriented metrics, such as Predictive Entropy and Least Confidence, correlated positively with FPR ($\rho \approx +0.47$ and $+0.48$, respectively). Given that higher Maximum Probability denotes lower uncertainty, this pattern was directionally consistent: classifiers with greater predictive uncertainty on a dataset tended to yield higher false positive rates. Similarly, AUC correlated negatively with Predictive Entropy (Spearman $\rho \approx -0.44$; Kendall $\tau \approx -0.32$), suggesting that more dispersed predictive distributions were associated with weaker discrimination. Conversely, Maximum Probability correlated positively with AUC ($\rho \approx +0.44$).

In contrast, the correlation with MCC was marginal and inconsistent across coefficients (Spearman $\rho \approx +0.13$; Pearson $r \approx +0.23$). Because MCC is a balanced metric reflecting the full confusion matrix, its weak correlation with mean UQ suggests that aggregate uncertainty is a weak proxy for balanced classification effectiveness. Precision, Recall, and F1 score likewise had negligible global correlations; notably, for the F1 score, all Spearman correlations with the five UQ metrics were near zero ($|\rho| \approx 0$).

Finding 1 (WPDP): Globally, correlations between UQ and performance in WPDP were generally weak, appearing consistent only for specific metric pairs. These results demonstrate that UQ is *not* a universal proxy for classifier performance in SDP; rather, its utility is highly contingent upon the choice of evaluation metric and the specific dataset context.

3.3.2 RQ1.2: Category-Level Alignment Between UQ and Performance. While Finding 1 showed that aggregate correlations between UQ and performance in WPDP were generally marginal, we next investigated whether this trend was maintained across classifier categories. Identifying specific categories that yield performance-correlated UQ is essential for the practical application of uncertainty estimates in model selection or performance assessment in SDP.

Experimental Design. We computed an alignment score between UQ and performance for each classifier and each performance metric (AUC, MCC, F1 score, Precision, Recall, FPR). The score was the mean Spearman correlation between dataset-level UQ values and dataset-level performance metric values across the five UQ metrics.

Result. Figure 3 illustrates the alignment scores between UQ and performance across the six evaluation metrics. We observed four key patterns. First, AUC alignment was positive: most classifiers showed moderate-to-strong alignment between UQ and AUC, led by neural networks and imbalance-aware ensembles (MLP 1 and EasyEnsemble: 0.76; MLP-Bagging: 0.75; RUSBoost: 0.70; Gaussian Process: 0.64), while only CART (-0.02) and SGDClassifier (0.00) were near zero. Second, FPR alignment was also positive and often strong (e.g., KNN 0.69, MLP 2 0.68, MLP 1 0.67, SVM Bagging 0.66), consistent with Finding 1, where UQ correlated most strongly with FPR and AUC. Third, alignment with threshold-dependent metrics (MCC, Precision, F1 score) was weaker and sometimes negative even for classifiers with strong AUC alignment (e.g., MLP 1 Precision

alignment -0.23 , KNN Precision -0.33), so a strong UQ-AUC signal did not imply a useful signal for threshold-based decisions. Fourth, the min-max range across the five UQ metrics was narrow for almost all classifiers (≤ 0.05), indicating that alignment conclusions were robust to the choice of UQ metric; in binary classification the five probability-based UQ metrics are near-monotone transforms of one another.

- Ensemble.** The ensemble category contained three imbalance-aware classifiers (RUSBoostClassifier, EasyEnsembleClassifier, BalancedRandomForestClassifier) and showed among the highest AUC alignment in the benchmark. EasyEnsembleClassifier had the largest AUC alignment (0.76), followed by RUSBoostClassifier (0.70) and BalancedRandomForestClassifier (0.58). The min-max ranges were narrow (≤ 0.06 for all three), confirming that their UQ signals were consistent across all five UQ metrics. This strong AUC alignment did not, however, carry over to MCC: EasyEnsemble (0.21) and RUSBoost (0.17) led, while BalancedRandomForest was near zero (-0.02). Thus, imbalance-aware ensembles provided a strong and consistent UQ signal for AUC but only a weak one for balanced classification performance.
- Linear Model.** PassiveAggressiveClassifier (0.48 for AUC) and Logistic Regression (0.40) showed moderate positive AUC alignment, while SGDClassifier was neutral (0.00). For MCC, all three were weak (PassiveAggressiveClassifier 0.08, Logistic Regression 0.02, SGDClassifier 0.01). The linear category therefore aligned with AUC but not with balanced classification performance. Differences in score generation and probability transformation affected how uncertainty metrics correlated with performance.
- Decision Tree.** CART had near-zero AUC alignment (-0.02) and weak alignment for the other metrics (MCC 0.06, Precision 0.18, FPR 0.06). It produced the most concentrated probabilities of any classifier (mean Maximum Probability 0.980, Predictive Entropy 0.048). This concentration left almost no dynamic range for UQ metrics, which reduced their capacity to correlate with any downstream performance metric. Its min-max range across the five UQ metrics was narrow (AUC: -0.03 to -0.01), so the near-zero alignment held across all five UQ metrics. CART was thus the clearest case of an uninformative UQ signal in the benchmark.
- Lazy Learning.** KNN showed moderate AUC alignment (0.34) and strong FPR alignment (0.69), but negative alignment for Precision (-0.33) and MCC (-0.11). Its distance-based probability estimates therefore produced UQ signals that tracked ranking and false-positive behavior but were inversely related to threshold-dependent precision.
- Probabilistic Model.** Gaussian Process Classifier (0.64 for AUC) showed the highest AUC alignment in this category, followed by Complement Naive Bayes (0.39), Naive Bayes (0.20), and QDA (0.14). Naive Bayes was unusual in aligning more strongly with MCC (0.39) and Precision (0.47) than with AUC, the opposite of most classifiers, whereas Complement Naive Bayes showed negative MCC (-0.10) and Precision (-0.20) alignment. These classifiers produced concentrated probabilities under distributional assumptions that are violated by software metrics (Naive Bayes mean Maximum Probability 0.979, QDA 0.968). The present analysis does not isolate whether the Gaussian Process pattern came from the kernel model, the probability transformation, or averaging across random seeds.
- Neural Network.** All three neural classifiers showed strong AUC alignment (MLP 1: 0.76, MLP-Bagging: 0.75, MLP 2: 0.62) and among the highest MCC alignment in the benchmark (MLP 1: 0.38, MLP-Bagging: 0.29, MLP 2: 0.24), with only Naive Bayes (0.39) ranking marginally higher on MCC. In contrast to the imbalance-aware ensembles, the neural classifiers thus produced UQ signals that aligned with both ranking (AUC) and

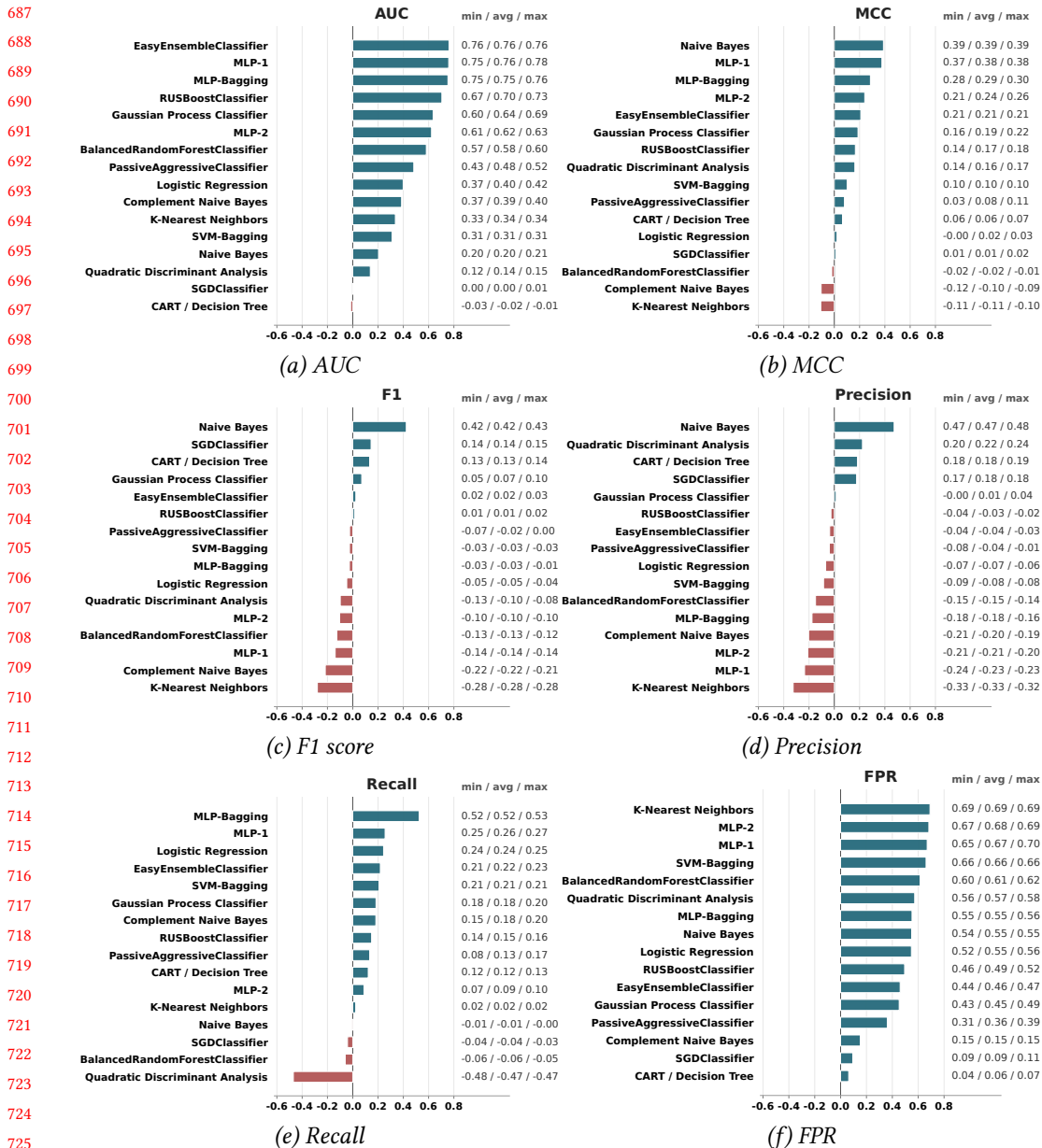


Fig. 3. Alignment scores between UQ and performance across six performance metrics. Each panel shows the alignment score (Spearman correlation between dataset-level UQ values and performance metric values, sign-adjusted so that higher is better) with the minimum, average, and maximum across the five UQ metrics displayed for each classifier. Classifiers are sorted by the average alignment score within each panel.

balanced (MCC) performance. The three architectures behaved consistently, indicating that the softmax probability mechanism yielded informative uncertainty signals regardless of depth or bagging.

- **SVM-Based.** SVM Bagging showed moderate AUC alignment (0.31), strong FPR alignment (0.66), but near-zero MCC (0.10) and Precision (-0.08) alignment. Despite its concentrated probability estimates (mean Maximum Probability 0.945), its margin-derived UQ signal still tracked ranking and false-positive behavior, though not threshold-dependent performance.

Finding 2 (WPDP): Alignment between UQ and performance varied more by target metric than by classifier category under WPDP. For AUC, most classifiers showed moderate-to-strong positive alignment, led by neural networks (MLP 1: 0.76, MLP-Bagging: 0.75) and imbalance-aware ensembles (EasyEnsemble: 0.76, RUSBoost: 0.70); only CART (-0.02) and SGDClassifier (0.00) were near zero. The same classifiers aligned more weakly, and sometimes negatively, with threshold-dependent metrics such as MCC and Precision. The alignment was consistent across the five UQ metrics for all classifiers (min–max ranges ≤ 0.10), reflecting that probability-based UQ metrics are near-monotone transforms of one another in binary classification. In practice, UQ should be interpreted as a metric-specific signal—reliable for AUC and FPR but not for threshold-dependent metrics such as MCC and F1 score—rather than as a property of the uncertainty metric alone.

3.3.3 *RQ1.3: Granularity-Level Correlation Between UQ and Performance.* While Finding 2 characterized how alignment between UQ and performance varied across classifier categories and target metrics, we next investigated whether it was also influenced by prediction granularity. SDP datasets typically use different granularities, such as class-level modules (AEEEM, PROMISE), function- or method-level modules (NASA), and file-level modules (ReLink). For UQ metrics to be effective in performance assessment, it is essential to determine whether their usefulness changes across prediction granularities.

Experimental Design. We grouped the 36 datasets into three granularities: class (AEEEM and PROMISE, 20 datasets), function (NASA, 12 datasets), and file (ReLink, 4 datasets). The three groups contained 20, 12, and 4 datasets, respectively, and we evaluated each group with all 16 classifiers. For each group, we computed Spearman correlations between each UQ metric and each performance metric per classifier across the datasets in that group. We then reported the mean correlation across the 16 classifiers with a one-sample t test for significance.

Result. Figure 4 illustrates the distribution of dataset-level mean uncertainty across the three granularity levels. While median uncertainty levels remained relatively consistent across groups, their variance differed substantially: the file-level ReLink group exhibited the widest dispersion, whereas the class-level and function/method-level groups demonstrated more concentrated distributions. This pattern suggests that the correlation between UQ and performance depends less on a uniform shift in confidence than on the degree of uncertainty variation across classifiers and datasets within a specific group.

- **Class Level.** At the class level, disaggregating the two dataset collections revealed distinct profiles. AEEEM remained weak: MaxProb vs. AUC was small and not significant ($\rho \approx 0.13$, n.s.), and among its UQ–performance correlations only FPR reached significance (MaxProb vs. FPR $\rho \approx -0.33$, $p < 0.01$). PROMISE was stronger: both MaxProb vs. AUC ($\rho \approx 0.33$, $p < 0.001$) and MaxProb vs. FPR ($\rho \approx -0.28$, $p < 0.001$) were significant, while correlations with F1 score and MCC stayed weak. Predictive Entropy mirrored these patterns with opposite signs (PROMISE Entropy vs. AUC $\rho \approx -0.32$). Mean class-level performance was moderate (MCC ≈ 0.30 , AUC ≈ 0.74). The divergence between AEEEM and PROMISE likely

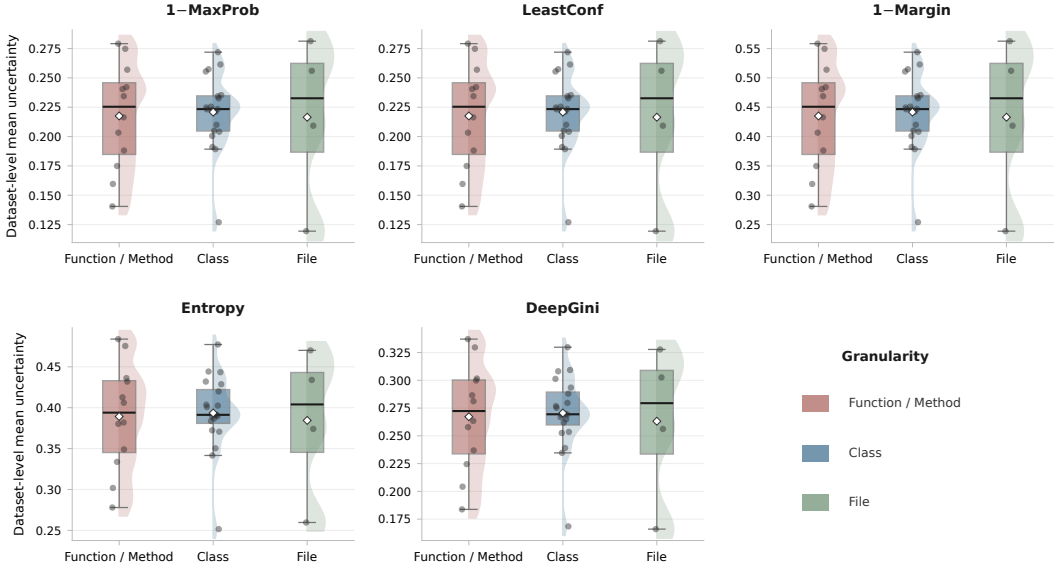


Fig. 4. **Uncertainty distribution by prediction granularity-aware dataset group.** Each panel shows the dataset-level mean of one UQ metric for class (AEEEM and PROMISE), function/method (NASA), and file (ReLink) datasets. Box plots show median and interquartile range; half violin plots show density; points are individual datasets; diamonds mark means. The file-level ReLink group has the widest uncertainty spread.

reflected differences in dataset composition: AEEEM comprises five projects from the Eclipse ecosystem with relatively homogeneous metric distributions, leaving limited cross-dataset variance for UQ to track, whereas PROMISE spans 15 projects with more diverse defect ratios and feature profiles, providing enough variability for a detectable UQ–performance correlation.

- Function Level.** At the function level (NASA), correlations were stronger than at the class level and split into two distinct patterns. Maximum Probability correlated positively with AUC ($\rho \approx 0.49$, $p < 0.001$), but negatively with F1 score ($\rho \approx -0.33$, $p < 0.001$), Precision ($\rho \approx -0.36$, $p < 0.001$), and FPR ($\rho \approx -0.66$, $p < 0.001$). The negative F1 score and Precision correlations indicated that higher-certainty classifiers tended to have worse threshold-dependent performance at the default cutoff; yet the large negative FPR correlation means higher confidence is correlated with fewer false positives. Entropy-oriented metrics mirrored these patterns (e.g., Predictive Entropy vs. FPR $\rho \approx 0.67$, $p < 0.001$). Mean function/method-level performance was the lowest of the three groups (MCC ≈ 0.26 , AUC ≈ 0.74). The opposing signs between ranking-based (AUC) and threshold-dependent (F1 score, Precision) metrics indicated that at the function level, classifier confidence improved ranking ability but did not translate into better decision-boundary performance at the default cutoff.
- File Level.** At the file level (ReLink), correlations between UQ and performance were the largest in the benchmark. Maximum Probability correlated positively with AUC ($\rho \approx 0.79$), MCC ($\rho \approx 0.78$), Precision ($\rho \approx 0.76$), F1 score ($\rho \approx 0.70$), and Recall ($\rho \approx 0.63$), and negatively with FPR ($\rho \approx -0.70$), all at $p < 0.001$. Predictive Entropy mirrored this pattern (e.g., Entropy vs. AUC $\rho \approx -0.75$, $p < 0.001$). Mean file-level performance was the highest of the three groups (MCC ≈ 0.51 , AUC ≈ 0.80). The strong correlations at the file level may be attributed to two factors: the ReLink datasets exhibited the widest uncertainty

dispersion among all groups (Figure 4), providing sufficient dynamic range for UQ metrics to differentiate dataset difficulty, and the coarser file-level granularity may have amplified differences in defect characteristics across datasets. This group contained only four datasets, so we treated the result as ReLink-specific evidence rather than as evidence that file-level prediction increases the correlation between UQ and performance.

These granularity-level results indicated that the strength of UQ–performance correlation increased with the heterogeneity of the underlying data: AEEEM (homogeneous Eclipse projects) showed the weakest signals, PROMISE and NASA were progressively stronger, and ReLink (the widest uncertainty dispersion) showed the strongest. Because granularity and dataset collection were coupled in this benchmark, these patterns should be interpreted as joint effects of both factors rather than as isolated granularity effects.

Finding 3 (WPDP): Correlations between UQ and performance differed across granularities and grew with dataset heterogeneity. At the class level, AEEEM showed no statistically significant UQ vs. AUC correlation ($\rho \approx 0.13$, n.s.), while PROMISE showed a significant positive correlation (MaxProb vs. AUC $\rho \approx 0.33$, $p < 0.001$). At the function level, MaxProb vs. AUC was stronger ($\rho \approx 0.49$, $p < 0.001$), but MaxProb vs. F1 score was negative ($\rho \approx -0.33$), with FPR showing the largest signal (MaxProb vs. FPR $\rho \approx -0.66$). At the file level, correlations between UQ and performance were the largest in the benchmark (MaxProb vs. AUC $\rho \approx 0.79$, MaxProb vs. F1 score $\rho \approx 0.70$, MaxProb vs. FPR $\rho \approx -0.70$).

3.4 RQ2: Does high predictive performance imply well-calibrated probabilities in within-project defect prediction?

RQ2 examined whether predictive performance served as evidence of calibration. We analyzed this question in two steps. First, RQ2.1 correlated six performance metrics with three calibration metrics across classifier and dataset observations to test whether high performance was associated with lower calibration error. Second, RQ2.2 compared calibration profiles across classifier categories to examine whether calibration errors differed by probability-generation mechanism. Together, these analyses treated predictive performance and calibration as separate evaluation dimensions: performance metrics describe how well classifiers rank or label modules, whereas calibration metrics describe whether predicted probabilities match empirical defect frequencies.

3.4.1 RQ2.1: Is High Performance Correlated with Low Calibration Error? We computed Spearman correlations between each performance metric (MCC, Precision, Recall, FPR, AUC, F1 score) and each calibration metric (ECE, NLL, Brier Score) across all classifier and dataset pairs. Figure 5 summarizes all 18 pairwise correlations. Each panel represents one comparison between performance and calibration across all classifier and dataset observations. The horizontal axis is a performance metric, and the vertical axis is a calibration error metric. Darker regions indicate where more observations concentrate, the red line shows the fitted trend, and the annotation gives Spearman ρ . Because ECE, NLL, and Brier Score are error metrics, lower values indicate better calibration. Thus, a negative correlation for MCC, AUC, Precision, Recall, or F1 score means that better performance tends to coincide with lower calibration error. For FPR, the interpretation reverses because lower FPR is better; a positive correlation means that higher false positive rates tend to coincide with larger calibration error.

- **MCC was a moderate indicator of calibration error.** MCC correlated with ECE at $\rho = -0.38$.
- **AUC was more strongly correlated with Brier Score than with ECE.** AUC vs. Brier Score reached $\rho = -0.58$, one of the largest correlations in the matrix. This was expected

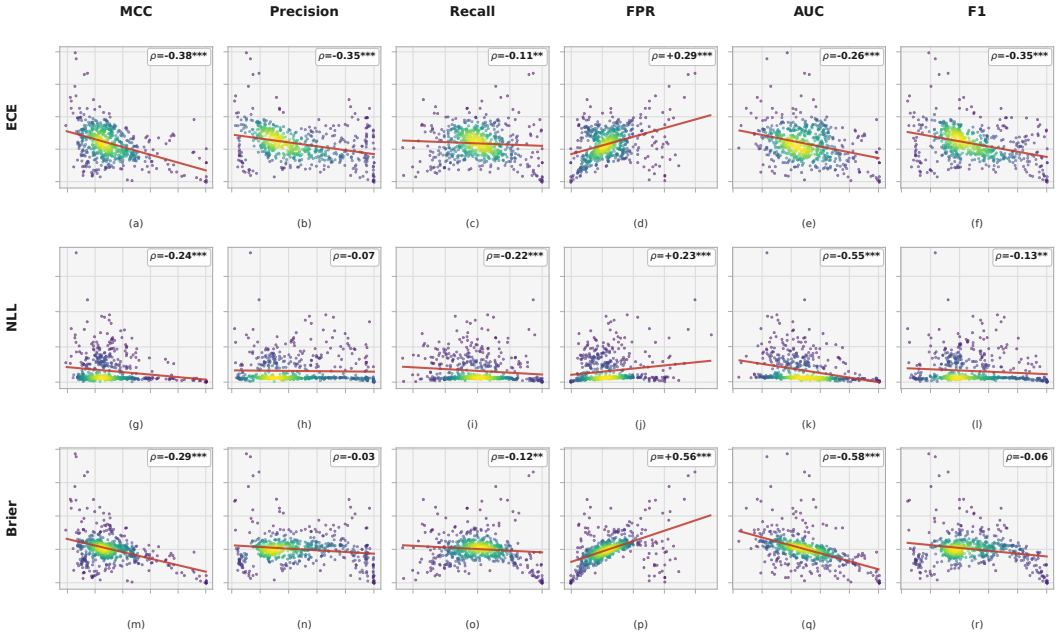


Fig. 5. **The correlation between performance and calibration across all metric pairs.** Rows correspond to calibration metrics (ECE, NLL, Brier); columns correspond to performance metrics (MCC, Precision, Recall, FPR, AUC, F1 score). Each panel shows all classifier and dataset observations; color indicates local density; the red line is a linear fit; annotation reports Spearman ρ . The largest correlations are AUC vs. Brier ($\rho = -0.58$), FPR vs. Brier ($\rho = +0.56$), AUC vs. NLL ($\rho = -0.55$), and MCC vs. ECE ($\rho = -0.38$).

because the Brier Score jointly measures calibration and discrimination, whereas ECE measures calibration alone. AUC vs. ECE was weaker ($\rho = -0.26$), indicating that higher discrimination does not necessarily translate into calibrated probabilities.

- **FPR was positively correlated with calibration error.** FPR vs. ECE was $\rho = +0.29$, and FPR vs. Brier Score was $\rho = +0.56$. Classifiers that produced many false positives tended to assign high positive-class probabilities to negative modules. This behavior increased the Brier Score and could increase ECE. This pattern is consistent with Finding 1, where UQ has its largest performance correlation with FPR.

Finding 4 (WPDP): Predictive performance and calibration were correlated but distinct. MCC correlated negatively with ECE ($\rho = -0.38$), and AUC correlated negatively with Brier Score ($\rho = -0.58$), so better-performing classifiers tended to have lower calibration error on average. However, the correlation was moderate, and many runs with high performance had large calibration error. FPR was positively correlated with calibration error ($\rho = +0.56$ vs. Brier), indicating that classifiers with high false positive rates also tended to have larger probability errors under the Brier Score. In practice, calibration needs to be checked before deployment.

3.4.2 RQ2.2: Calibration by Classifier Category. Finding 4 showed that performance and calibration had moderate correlations. We next examined whether calibration itself varied systematically across classifier categories.

For each classifier, we computed mean ECE, NLL, and Brier Score across all datasets. We grouped classifiers according to the categories listed in Table 1. Figure 6 shows the calibration profile of

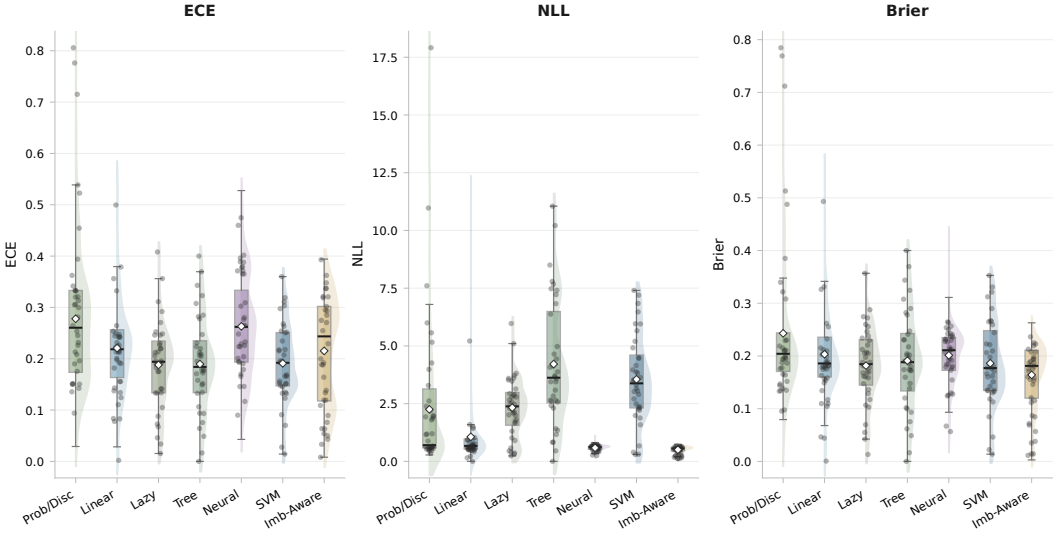


Fig. 6. **Calibration distribution by classifier category.** Panels show ECE, NLL, and Brier Score. Each panel displays the distribution of dataset-level calibration errors for all valid classifier and dataset runs within a category: half violin density, box plot, individual points, and mean marker. The categories follow the order of mean ECE. Low ECE does not always coincide with low NLL: KNN and CART have low ECE but high NLL due to occasional extreme probability errors.

each classifier category across the three metrics. Each panel represents one calibration error metric across classifier categories. The horizontal axis lists the classifier categories, and the vertical axis reports the calibration error value. The violin shape shows the distribution of observations within a category, the box summarizes the median and interquartile range, the points show individual classifier and dataset runs, and the mean marker shows the category average. Because ECE, NLL, and Brier Score are all error metrics, lower positions indicate better calibration in each panel. Differences across panels are important: a category can have low ECE but high NLL, which means average bin error is small while occasional extreme probability errors remain large.

- Probabilistic Model: the largest aggregate calibration error.** This category had the largest aggregate calibration error (ECE = 0.278, NLL = 2.166, Brier Score = 0.242). Naive Bayes and QDA were the main contributors: Naive Bayes had ECE = 0.302, NLL = 4.081, and Brier Score = 0.295; QDA had ECE = 0.299, NLL = 3.473, and Brier Score = 0.291. Gaussian Process Classifier had lower calibration error (ECE = 0.239, NLL = 0.538) than the other members of this category, but its calibration remained weaker than that of the Linear Model and Ensemble categories. The independence assumptions in Naive Bayes and the distributional assumptions in QDA can push probability mass toward extremes. This pattern was consistent with their larger calibration errors on these SDP data.
- Linear Model: moderate calibration error with internal heterogeneity.** This category showed the widest internal spread. Logistic Regression had lower calibration error (ECE = 0.192, NLL = 0.545), but SGDClassifier (ECE = 0.237, NLL = 1.815) and PassiveAggressiveClassifier (ECE = 0.242, NLL = 0.763) increased the category average. These two classifiers also had high Recall in the category (0.625 and 0.610). This suggests that recall-oriented linear methods can produce positive-class probability errors.

- **Lazy Learning: low ECE but high NLL.** KNN had mean ECE = 0.183 and Brier Score = 0.182, but its NLL was 2.212. Its distance-based probability estimates made bin-level averages appear low while occasional extreme probability errors remained large under log loss.
- **Decision Tree: low ECE but high NLL.** CART had mean ECE = 0.195 and Brier Score = 0.196, but NLL reached 4.423, the highest among all 16 classifiers. Its leaf node frequency probabilities were nearly binary, causing extreme log loss penalties on misclassified instances.
- **Neural Network: moderate calibration error with internal differentiation.** This category (MLP 1, MLP 2, MLP Bagging) had mean ECE = 0.255, NLL = 0.576, and Brier Score = 0.197. MLP 1 and MLP 2 produced relatively high bin-level calibration error even though their NLL remained moderate. MLP Bagging had higher ECE (0.246) but low NLL (0.546) and Brier Score (0.184). It therefore had a different calibration profile from the single MLP classifiers. The three neural classifiers thus displayed heterogeneous calibration behavior despite sharing the same softmax probability mechanism.
- **SVM-Based: extreme errors under log loss.** SVM Bagging had low ECE (0.199) but high NLL (3.717), indicating occasional extreme probability errors. Its probability estimates could produce extreme values near 0 or 1 that inflated NLL but averaged out in bin-level ECE.
- **Ensemble: the lowest aggregate NLL and Brier Score.** This category (BalancedRandomForest, EasyEnsemble, RUSBoost) had mean ECE = 0.214, NLL = 0.509, and Brier Score = 0.164. BalancedRandomForest had the lowest ECE (0.100) and also had the lowest Brier Score (0.113). EasyEnsemble and RUSBoost had higher ECE (0.266 and 0.277), but their NLL and Brier values remained moderate.

Finding 5 (WPDP): Calibration error varied by classifier category. For instance, the Ensemble category had the lowest aggregate NLL/Brier profile, with BalancedRandomForest showing the lowest individual ECE (0.100). SVM-Based classifiers had low ECE but high NLL because extreme probability estimates inflated log-loss penalties.

3.5 RQ3: Do relationships among UQ, performance, and calibration persist under cross-project defect prediction?

RQ1 and RQ2 examined WPDP, where training and test data came from the same project. CPDP supports prediction when a new project lacks historical defect data. RQ3 asked how UQ related to performance and calibration under CPDP, and whether those correlations changed from WPDP to CPDP.

We evaluated CPDP using a within-collection Direct Transfer protocol [3, 50, 51, 83]. Within each included collection, one dataset served as the source project, and a different dataset from the same collection served as the target project. We aligned source and target features by common feature names and retained only source-target pairs with sufficient feature overlap. We excluded ReLink because its four datasets formed two feature-incompatible subgroups. The two subgroups had no overlapping features, and each subgroup contained only two datasets, leaving too few source-target pairs for stable CPDP analysis. The CPDP experiments therefore focused on AEEEM (5 datasets), NASA (12 datasets), and PROMISE (15 datasets).

3.5.1 RQ3.1: How Is UQ Related to Performance and Calibration Under CPDP?. Experimental Design. We generated CPDP results using Direct Transfer for all ordered source → target permutations within AEEEM, NASA, and PROMISE. The evaluation used the 16 classifiers for all ordered source-target pairs. For each dataset collection, we computed the Spearman correlation between

each UQ metric and each CPDP target metric. The target metrics included six performance metrics (MCC, Precision, Recall, FPR, AUC, and F1 score) and three calibration metrics (ECE, NLL, and Brier Score).

Result. The CPDP correlations showed two patterns. This subsection reads the CPDP correlations directly; RQ3.2 then compares them with WPDP.

- **UQ and performance under CPDP.** Performance metric correlations under CPDP remained specific to the metric and dataset collection rather than global. In NASA, the largest visible performance correlations involved AUC: MaxProb vs. AUC was negative ($\rho = -0.342$), while entropy-based AUC correlations were positive. AEEEM and PROMISE did not show a uniform CPDP performance pattern across all UQ metrics and performance metrics. Thus, CPDP did not turn UQ into a general performance signal.
- **UQ and calibration under CPDP.** Calibration metric correlations were more sensitive to the dataset collection. In AEEEM, MaxProb vs. Brier reached $\rho = 0.328$ under CPDP. In PROMISE, MaxProb vs. ECE reached $\rho = 0.234$, and the mean absolute correlation for calibration pairs was 0.482 under CPDP. NASA showed weaker CPDP calibration correlations: its two Brier Score cases were close to zero. These results indicated that CPDP can change how confidence-based UQ relates to calibration error.

Finding 6 (CPDP): Under CPDP, UQ was related to both performance and calibration, but the correlation was conditional on the metric and dataset collection. Performance relations concentrated in selected metric pairs rather than all performance metrics. Calibration relations showed stronger collection-specific behavior, especially in AEEEM and PROMISE. For SDP practice, CPDP evaluation should report UQ correlations with both performance metrics and calibration metrics.

3.5.2 RQ3.2: How Do UQ Relationships Change from WPDP to CPDP? Experimental Design.

Using the same (UQ, target metric) correlations, we compared the CPDP Spearman correlation with the corresponding WPDP Spearman correlation within each dataset collection. We used *correlation direction reversal* as a signal of transfer stability. We counted a reversal when the WPDP and CPDP correlations pointed in opposite directions.

Result. Table 4 summarizes the comparison between WPDP and CPDP numerically. Reading the table along the WPDP-to-CPDP axis separates what transferred from what did not: UQ–performance correlations were largely stable in direction, with no performance reversals in NASA or PROMISE, whereas UQ–calibration correlations reversed direction in a collection-dependent way. Three collection-level changes made this split concrete.

Table 4. **Summary of UQ correlation changes from WPDP to CPDP.** Each row summarizes 45 pairs formed by five UQ metrics and nine target metrics. The reversal column reports total correlation direction reversals, with performance/calibration counts in parentheses.

Collection	WPDP $ \rho $	CPDP $ \rho $	Rev. (Perf/Cal)	Main pattern
AEEEM	0.251	0.222	15 (5/10)	Calibration changes dominate, with additional performance reversals.
NASA	0.227	0.279	2 (0/2)	Mostly stable; only weak Brier Score reversals appear.
PROMISE	0.141	0.234	5 (0/5)	Calibration direction reversals dominate, especially for ECE.

- **AEEEM: calibration correlations were unstable under CPDP.** AEEEM had 15 descriptive direction reversals out of 45 UQ and target metric pairs. Ten involved calibration

metrics; for example, MaxProb vs. Brier changed from $\rho = -0.177$ in WPDP to $\rho = 0.328$ under CPDP. The mean absolute correlation decreased from 0.251 in WPDP to 0.222 under CPDP. This indicated that, in AEEEM, the correlation between probability concentration and calibration error changed under transfer.

- **NASA: correlations increased with only weak Brier Score reversals.** NASA had 2 direction reversals, both weak calibration pairs (Predictive Entropy/DeepGini vs. Brier Score) where the WPDP correlation was slightly above 0.1 and the CPDP correlation was close to zero. The mean absolute correlation increased from 0.227 in WPDP to 0.279 under CPDP. The largest changes involved performance metrics: the MaxProb vs. AUC correlation became more negative (-0.070 to -0.342), and the entropy-based AUC correlations became more positive. Thus, CPDP did not remove the NASA correlations between UQ and target metrics. Several correlations increased, while only the two weak Brier Score pairs crossed the reversal criterion.
- **PROMISE: calibration direction reversals concentrated in calibration metrics.** PROMISE had 5 direction reversals, all involving calibration metrics. For example, MaxProb vs. ECE changed from $\rho = -0.208$ in WPDP to $\rho = 0.234$ under CPDP. The mean absolute correlation increased from 0.141 in WPDP to 0.234 under CPDP, mainly because calibration pairs increased from 0.271 to 0.482. This indicated that CPDP can reverse the correlation between confidence and calibration error even when correlations between UQ and performance remain weak.

Finding 7 (CPDP): UQ correlations changed from WPDP to CPDP in a collection-dependent manner. AEEEM and PROMISE showed calibration metric direction reversals, while NASA showed only two weak Brier Score reversals. Practitioners should not transfer the direction and magnitude of UQ correlations from WPDP to CPDP without target collection checks.

4 Discussion and Implications

4.1 Summary of Findings

The seven findings showed that probability-based UQ is informative in SDP, but only under explicit evaluation conditions. The meaning of a UQ score depends on the target metric, classifier category, dataset collection, prediction granularity, and transfer setting. Table 5 summarizes this interpretation across the three empirical dimensions examined in this study. The first two dimensions are established under WPDP, whereas the third examines how these relationships transfer to CPDP.

This summary clarifies three points about UQ in SDP, each corresponding to a knowledge gap identified in Section 3.1. First, regarding Gap 1, UQ has metric-specific implications rather than serving as a general measure of classifier quality. The global WPDP analysis showed consistent correlations mainly for FPR and AUC. The category and dataset analyses further showed that the same UQ metric can become weak, inverted, or collection-specific under different settings. Thus, a high or low uncertainty score is not sufficient on its own. It needs to be interpreted together with the performance metric and the dataset collection.

Second, regarding Gap 2, calibration remains a separate evaluation dimension that cannot be inferred from predictive performance alone. Finding 4 showed that better predictive performance was correlated with lower calibration error on average. However, the correlation was moderate, and many high-performing runs still showed large calibration error. Finding 5 further showed that calibration profiles varied by classifier category. For example, CART and SVM Bagging had low ECE but high NLL because extreme probability estimates created large log loss penalties. Predicted

Table 5. Summary of findings by analytical dimension and evidence level.

Dimension	Findings	Key Evidence	Level
UQ and performance	Findings 1, 2, 3 (WPDP)	UQ was strongly correlated with FPR and AUC. Correlations with MCC, Precision, Recall, and F1 score were weak or unstable. Alignment also varied across classifier categories and dataset groups	Global, category, dataset collection
Performance and calibration	Findings 4, 5 (WPDP)	Predictive performance had a moderate correlation with calibration error, but high discrimination could coexist with large ECE, NLL, or Brier Score. Calibration profiles differed by classifier category	Metric, category
Cross-project transfer	Findings 6, 7 (CPDP)	UQ remained correlated with selected performance and calibration metrics under CPDP. Several correlations changed magnitude or direction, especially calibration correlations in AEEEM and PROMISE	Dataset collection

probabilities in SDP therefore require calibration evaluation when they are analyzed as probabilities rather than only as class labels.

Third, regarding Gap 3, cross-project transfer changes how UQ should be read. RQ3 showed that UQ remained related to selected performance and calibration metrics under CPDP, but these relations were collection-dependent. AEEEM and PROMISE showed calibration direction reversals, while NASA showed only two weak Brier Score reversals. WPDP evidence is therefore useful as an initial reference, but it does not determine how UQ behaves after transfer to a target collection.

These results complement prior UQ studies based on Bayesian neural networks and deep ensembles [16, 35] by showing that probability outputs from classical SDP classifiers also provide measurable but context-dependent implications for uncertainty interpretation. They also relate recent SDP calibration studies [61, 62] to foundational CPDP research [51, 83] and broader calibration work [9, 70]. Together, these findings indicate that calibration, predictive performance, and transfer behavior should be reported as separate evaluation dimensions.

4.2 Implications for Evaluation and Reporting

The findings suggest the following practices for SDP studies that use probability-based UQ.

1. Do not use UQ as a proxy for MCC or F1 score. UQ showed consistent correlations with FPR and AUC, but its correlations with MCC, F1 score, Precision, and Recall were weak or unstable across classifiers and dataset collections (Findings 1 and 3). In practice, MCC and F1 score should be measured directly rather than inferred from uncertainty scores, because UQ does not reliably reflect these metrics.

2. Select UQ-informative classifiers deliberately. UQ signals were informative for AUC and FPR across most classifiers, but were weak for threshold-dependent metrics and absent for a few classifiers even on AUC (Finding 2). Neural networks (MLP 1, MLP-Bagging) and imbalance-aware ensembles (EasyEnsemble, RUSBoost) produced strong and consistent UQ–AUC alignment (≥ 0.70 , with narrow min–max ranges), whereas CART (−0.02) and SGDClassifier (0.00) produced near-zero alignment. Practitioners who intend to use UQ for model selection or performance monitoring

should rely on it only for ranking-oriented metrics (AUC and FPR) and should avoid classifiers with degenerate probability outputs (e.g., CART, SGDClassifier), whose UQ carries no usable signal.

3. Report multiple calibration metrics, not ECE alone. CART and SVM Bagging had low ECE but high NLL because their extreme probability estimates incurred large log-loss penalties that bin-averaged ECE masked (Finding 5). Future SDP studies that analyze predicted probabilities should report at least two calibration metrics (e.g., ECE together with NLL or Brier Score) to avoid misleading conclusions about probability quality.

4. Treat calibration as independent of discrimination. High AUC or MCC does not guarantee well-calibrated probabilities (Finding 4). Practitioners who use predicted probabilities for inspection prioritization, threshold tuning, or cost-sensitive decision-making should evaluate calibration independently, because a classifier that ranks modules correctly may still assign inaccurate defect probabilities.

5. Account for dataset collection effects when interpreting UQ. Even within the same prediction granularity, UQ profiles differed across collections: AEEEM showed near-zero UQ-performance signals while PROMISE showed weak but detectable correlations (Finding 3). Future UQ studies should report results separately for each dataset collection, because aggregate correlations can mask opposing patterns across collections (e.g., near-zero signals in AEEEM vs. weak positive signals in PROMISE).

6. Re-evaluate UQ-calibration correlations after cross-project transfer. Under CPDP, several calibration correlations changed direction in AEEEM and PROMISE, while NASA largely preserved its WPDP patterns (Findings 6 and 7). CPDP practitioners should not assume that UQ-calibration correlations observed in within-project settings transfer unchanged; re-evaluation on the target collection is necessary before using UQ for deployment decisions.

5 Threats to Validity

Internal Validity. The main internal threat is leakage from preprocessing and repeated evaluation. The experimental protocol separates training and evaluation before scaling. In WPDP, we performed the training and test split before scaling; we fitted the scaler on the training subset and then applied it to the test subset. In CPDP, we fitted the scaler on the source project and then applied it to the target project. This design keeps target project information out of model fitting and preprocessing. To reduce random variation from training and test splits and stochastic classifiers, all WPDP and CPDP experiments used the repeated run protocol described in Section 3.2.4. The correlations therefore summarized repeated observations rather than a single split or seed.

External Validity. The benchmark includes 36 datasets from four collections (AEEEM, NASA, PROMISE, and ReLink), spanning multiple programming languages and application domains. However, all datasets come from public repositories commonly used in SDP research. The results may not generalize to proprietary industrial systems with different feature distributions, labeling practices, or defect reporting processes. The classifier set covers 16 classical classifiers that are common in SDP studies, but it does not include newer code representation models or pretrained language models for code.

Another external threat is the coupling between prediction granularity and dataset collection. In this benchmark, class-level datasets come from AEEEM and PROMISE, function/method-level datasets from NASA, and file-level datasets from ReLink. The granularity-aware analysis may therefore reflect collection-specific data collection, feature definitions, or label quality rather than prediction granularity alone. We addressed this threat by interpreting Finding 3 as collection and granularity heterogeneity rather than as a causal effect of granularity. The CPDP analysis is also limited to AEEEM, NASA, and PROMISE because ReLink lacks sufficient feature-compatible source-target pairs under the within-collection protocol. CPDP conclusions therefore apply to

feature-compatible collection-level transfers and should not be generalized to file-level ReLink transfer.

Construct Validity. The UQ measures in this study are five standard scores derived from predictive probabilities. In binary classification, these scores are closely related functions of the predicted class probabilities, so we do not interpret them as five fully independent uncertainty constructs. For classifiers without native probabilistic outputs, probability estimates depend on post hoc transformations or model-specific scoring functions, which can affect calibration behavior. The calibration metrics (ECE, NLL, and Brier Score) also summarize different aspects of probability error. ECE reports average bin-level error, NLL emphasizes extreme probability errors, and Brier Score measures squared probability error. As a result, the study characterizes dataset-level relationships among UQ, performance, and calibration rather than instance-level uncertainty explanations. Different binning schemes for ECE may also change absolute ECE values, although the paper focuses mainly on relationships and category-level patterns.

6 Conclusion

This study set out to clarify what probability-based uncertainty quantification means in software defect prediction. Across 16 representative classifiers and 36 benchmark datasets, we examined relationships among UQ, predictive performance, and probability calibration. We also tested whether these relationships changed under CPDP using 32 feature-compatible datasets from AEEEM, NASA, and PROMISE.

The findings indicate that UQ is informative only when its evaluation context is explicit. In WPDP, UQ–performance correlations were weak at the global level except for selected metric pairs, especially FPR and AUC. These correlations also varied across classifier categories and granularity-aware dataset groups. Most classifiers showed moderate-to-strong AUC alignment, led by neural networks and imbalance-aware ensembles, whereas only CART and SGDClassifier showed near-zero alignment. Predictive performance and calibration were related but not interchangeable. High discrimination could coexist with large calibration error, and classifier categories differed across ECE, NLL, and Brier Score. Under CPDP, UQ remained related to selected performance and calibration metrics. However, changes from WPDP to CPDP were collection-dependent. AEEEM and PROMISE showed calibration direction reversals, whereas NASA showed only two weak Brier Score reversals.

These results refine how probability scores should be reported in SDP studies. UQ should be linked to the specific performance metric it is intended to inform, and MCC or F1 score should not be inferred from uncertainty scores. Classifier choice matters: most classifiers provide informative UQ signals for AUC and FPR, but a few with degenerate probabilities (CART, SGDClassifier) do not. Calibration should be evaluated with multiple metrics and independently of discrimination. Results should be reported at the collection level, and UQ–calibration correlations should be re-evaluated after cross-project transfer. These conclusions are bounded by the classical probability-output classifiers and public tabular datasets studied here. Future work can examine whether the same relationship patterns hold for Bayesian neural networks, deep ensembles, pretrained code representations, and other modern feature spaces.

Acknowledgments

This work in this paper is partly supported by the Macau University of Science and Technology Faculty Research Grant (FRG) (No. FRG-26-030-FIE).

References

- [1] Moloud Abdar, Farhad Pourpanah, Sadiq Hussain, Dana Rezazadegan, Li Liu, Mohammad Ghavamzadeh, Paul Fieguth, Xiaochun Cao, Abbas Khosravi, U Rajendra Acharya, et al. 2021. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information fusion* 76 (2021), 243–297.
- [2] Amritanshu Agrawal and Tim Menzies. 2018. Is "better data" better than "better data miners"? on the benefits of tuning SMOTE for defect prediction. In *Proceedings of the 40th International Conference on Software engineering*. 1050–1061.
- [3] Sousuke Amasaki. 2018. Cross-version defect prediction using cross-project defect prediction approaches: Does it work?. In *Proceedings of the 14th International Conference on predictive models and data analytics in software engineering*. 32–41.
- [4] Rok Blagus and Lara Lusa. 2013. SMOTE for high-dimensional class-imbalanced data. *BMC bioinformatics* 14, 1 (2013), 106.
- [5] David Bowes, Tracy Hall, and Jean Petrić. 2018. Software defect prediction: do different classifiers find the same defects? *Software Quality Journal* 26, 2 (2018), 525–552.
- [6] Leo Breiman. 1996. Bagging predictors. *Machine learning* 24, 2 (1996), 123–140.
- [7] Gerardo Canfora, Andrea De Lucia, Massimiliano Di Penta, Rocco Oliveto, Annibale Panichella, and Sebastiano Panichella. 2015. Defect prediction as a multiobjective optimization problem. *Software Testing, Verification and Reliability* 25, 4 (2015), 426–459.
- [8] Cagatay Catal and Banu Diri. 2009. A systematic review of software fault prediction studies. *Expert systems with applications* 36, 4 (2009), 7346–7354.
- [9] Andrea Dal Pozzolo, Olivier Caelen, Reid A Johnson, and Gianluca Bontempi. 2015. Calibrating Probability with Undersampling for Unbalanced Classification.. In *SSCI*. 159–166.
- [10] Marco D'Ambros, Michele Lanza, and Romain Robbes. 2010. An extensive comparison of bug prediction approaches. In *2010 7th IEEE working conference on mining software repositories (MSR 2010)*. IEEE, 31–41.
- [11] Joost CF De Winter, Samuel D Gosling, and Jeff Potter. 2016. Comparing the Pearson and Spearman correlation coefficients across distributions and sample sizes: A tutorial using simulations and empirical data. *Psychological methods* 21, 3 (2016), 273.
- [12] Ran El-Yaniv et al. 2010. On the Foundations of Noise-free Selective Classification. *Journal of Machine Learning Research* 11, 5 (2010).
- [13] T Elhassan, M Aljurf, et al. 2016. Classification of imbalance data using totem link (t-link) combined with random under-sampling (rus) as a data reduction method. *Global J Technol Optim S* 1, S1 (2016).
- [14] Yuanrui Fan, Xin Xia, Daniel Alencar Da Costa, David Lo, Ahmed E Hassan, and Shanping Li. 2019. The impact of mislabeled changes by szz on just-in-time defect prediction. *IEEE transactions on software engineering* 47, 8 (2019), 1559–1586.
- [15] Yang Feng, Qingkai Shi, Xinyu Gao, Jun Wan, Chunrong Fang, and Zhenyu Chen. 2020. Deepgini: prioritizing massive tests to enhance the robustness of deep neural networks. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 177–188.
- [16] Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*. PMLR, 1050–1059.
- [17] Baljinder Ghotra, Shane McIntosh, and Ahmed E Hassan. 2015. Revisiting the impact of classification techniques on the performance of defect prediction models. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 789–800.
- [18] W Brier Glenn et al. 1950. Verification of forecasts expressed in terms of probability. *Monthly weather review* 78, 1 (1950), 1–3.
- [19] Tilmann Gneiting and Adrian E Raftery. 2007. Strictly proper scoring rules, prediction, and estimation. *Journal of the American statistical Association* 102, 477 (2007), 359–378.
- [20] Lina Gong, Haoxiang Zhang, Jingxuan Zhang, Mingqiang Wei, and Zhiqiu Huang. 2022. A comprehensive investigation of the impact of class overlap on software defect prediction. *IEEE transactions on software engineering* 49, 4 (2022), 2440–2458.
- [21] Somya R Goyal. 2025. Current Trends in Class Imbalance Learning for Software Defect Prediction. *IEEE Access* (2025).
- [22] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. 2017. On calibration of modern neural networks. In *International conference on machine learning*. PMLR, 1321–1330.
- [23] Tracy Hall, Sarah Beecham, David Bowes, David Gray, and Steve Counsell. 2011. A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering* 38, 6 (2011), 1276–1304.
- [24] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. 2005. Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning. In *International conference on intelligent computing*. Springer, 878–887.

- [25] Ahmed E Hassan. 2009. Predicting faults using the complexity of code changes. In *2009 IEEE 31st international conference on software engineering*. IEEE, 78–88.
- [26] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. 2008. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*. Ieee, 1322–1328.
- [27] Wenchong He, Zhe Jiang, Tingsong Xiao, Zelin Xu, and Yukun Li. 2025. A survey on uncertainty quantification methods for deep learning. *Comput. Surveys* (2025).
- [28] Dan Hendrycks and Kevin Gimpel. 2016. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *arXiv preprint arXiv:1610.02136* (2016).
- [29] Eyke Hüllermeier and Willem Waegeman. 2021. Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine learning* 110, 3 (2021), 457–506.
- [30] Xiaolin Ju, Yi Cao, Xiang Chen, Lina Gong, Vaskar Chakma, and Xin Zhou. 2025. JIT-CF: Integrating contrastive learning with feature fusion for enhanced just-in-time defect prediction. *Information and Software Technology* 182 (2025), 107706.
- [31] Marian Jureczko and Lech Madeyski. 2010. Towards identifying software project clusters with regard to defect prediction. In *Proceedings of the 6th international conference on predictive models in software engineering*. 1–10.
- [32] Maurice G Kendall. 1938. A new measure of rank correlation. *Biometrika* 30, 1-2 (1938), 81–93.
- [33] Sunghun Kim, Hongyu Zhang, Rongxin Wu, and Liang Gong. 2011. Dealing with noise in defect prediction. In *Proceedings of the 33rd international conference on software engineering*. 481–490.
- [34] Ludmila I Kuncheva and Christopher J Whitaker. 2003. Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine learning* 51, 2 (2003), 181–207.
- [35] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems* 30 (2017).
- [36] Issam H Laradji, Mohammad Alshayeb, and Lahouari Ghouti. 2015. Software defect prediction using ensemble learning on selected features. *Information and Software Technology* 58 (2015), 388–402.
- [37] Stefan Lessmann, Bart Baesens, Christophe Mues, and Swantje Pietsch. 2008. Benchmarking classification models for software defect prediction: A proposed framework and novel findings. *IEEE transactions on software engineering* 34, 4 (2008), 485–496.
- [38] David D Lewis. 1995. A sequential algorithm for training text classifiers: Corrigendum and additional data. In *Acm sigir forum*, Vol. 29. ACM New York, NY, USA, 13–19.
- [39] Ke Li, Zilin Xiang, Tao Chen, Shuo Wang, and Kay Chen Tan. 2020. Understanding the automated parameter optimization on transfer learning for cross-project defect prediction: an empirical study. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 566–577.
- [40] Zhiqiang Li, Xiao-Yuan Jing, and Xiaoke Zhu. 2018. Progress on approaches to software defect prediction. *Iet Software* 12, 3 (2018), 161–175.
- [41] Wangshu Liu, Ye Yue, Xiang Chen, Qing Gu, Pengzhan Zhao, Xuejun Liu, and Jianjun Zhao. 2024. SeDPKG: Semi-supervised software defect prediction with graph representation learning and knowledge distillation. *Information and Software Technology* 174 (2024), 107510.
- [42] Ana C Lorena, Luis PF Garcia, Jens Lehmann, Marcilio CP Souto, and Tin Kam Ho. 2019. How complex is your classification problem? a survey on measuring classification complexity. *ACM Computing Surveys (CSUR)* 52, 5 (2019), 1–34.
- [43] Wei Ma, Mike Papadakis, Anestis Tsakmalis, Maxime Cordy, and Yves Le Traon. 2021. Test selection for deep learning systems. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 2 (2021), 1–22.
- [44] Faseeha Matloob, Taher M Ghazal, Nasser Taleb, Shabib Aftab, Munir Ahmad, Muhammad Adnan Khan, Sagheer Abbas, and Tariq Rahim Soomro. 2021. Software defect prediction using ensemble learning: A systematic literature review. *IEEE Access* 9 (2021), 98754–98771.
- [45] Tim Menzies, Jeremy Greenwald, and Art Frank. 2007. Data mining static code attributes to learn defect predictors. *IEEE transactions on software engineering* 33, 1 (2007), 2–13.
- [46] Tim Menzies, Zach Milton, Burak Turhan, Bojan Cukic, Yue Jiang, and Ayse Bener. 2010. Defect prediction from static code features: current results, limitations, new approaches. *Automated Software Engineering* 17, 4 (2010), 375–407.
- [47] Matthias Minderer, Josip Djolonga, Rob Romijnders, Frances Hubis, Xiaohua Zhai, Neil Houlsby, Dustin Tran, and Mario Lucic. 2021. Revisiting the calibration of modern neural networks. *Advances in neural information processing systems* 34 (2021), 15682–15694.
- [48] Raimund Moser, Witold Pedrycz, and Giancarlo Succi. 2008. A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction. In *Proceedings of the 30th international conference on Software engineering*. 181–190.

- [49] Nachiappan Nagappan and Thomas Ball. 2005. Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th international conference on Software engineering*. 284–292.
- [50] Jaechang Nam and Sunghun Kim. 2015. Heterogeneous defect prediction. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*. 508–519.
- [51] Jaechang Nam, Sinno Jialin Pan, and Sunghun Kim. 2013. Transfer defect learning. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 382–391.
- [52] Alexandru Niculescu-Mizil and Rich Caruana. 2005. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd international conference on Machine learning*. 625–632.
- [53] Jeremy Nixon, Michael W Dusenberry, Linchuan Zhang, Ghassen Jerfel, and Dustin Tran. 2019. Measuring calibration in deep learning.. In *CVPR workshops*, Vol. 2.
- [54] Safa Omri and Carsten Sinz. 2020. Deep learning for software defect prediction: A survey. In *Proceedings of the IEEE/ACM 42nd international conference on software engineering workshops*. 209–214.
- [55] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, David Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. 2019. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. *Advances in neural information processing systems* 32 (2019).
- [56] Muhammed Maruf Öztürk. 2017. Which type of metrics are useful to deal with class imbalance in software defect prediction? *Information and Software Technology* 92 (2017), 17–29.
- [57] John Platt et al. 1999. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in large margin classifiers* 10, 3 (1999), 61–74.
- [58] Shaoming Qiu, Bicong E, and Jingjie He. 2025. Features extraction and fusion by attention mechanism for software defect prediction. *PloS one* 20, 4 (2025), e0320808.
- [59] Tobias Scheffer, Christian Decomain, and Stefan Wrobel. 2001. Active hidden markov models for information extraction. In *International symposium on intelligent data analysis*. Springer, 309–318.
- [60] Patrick Schober, Christa Boer, and Lothar A Schwarte. 2018. Correlation coefficients: appropriate use and interpretation. *Anesthesia & analgesia* 126, 5 (2018), 1763–1768.
- [61] Xhulja Shahini, Jone Bartel, and Klaus Pohl. 2025. On the calibration of Just-in-time Defect Prediction. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 14–26.
- [62] Xhulja Shahini, Andreas Metzger, and Klaus Pohl. 2024. An Empirical Study on Just-in-time Conformal Defect Prediction. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 88–99.
- [63] Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal* 27, 3 (1948), 379–423.
- [64] Martin Shepperd, Qinbao Song, Zhongbin Sun, and Carolyn Mair. 2013. Data quality: Some comments on the nasa software defect datasets. *IEEE Transactions on software engineering* 39, 9 (2013), 1208–1215.
- [65] Michael R Smith, Tony Martinez, and Christophe Giraud-Carrier. 2014. An instance level analysis of data complexity. *Machine learning* 95, 2 (2014), 225–256.
- [66] Charles Spearman. 1961. The proof and measurement of association between two things. (1961).
- [67] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2016. Automated parameter optimization of classification techniques for defect prediction models. In *Proceedings of the 38th international conference on software engineering*. 321–332.
- [68] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2016. An empirical comparison of model validation techniques for defect prediction models. *IEEE Transactions on Software Engineering* 43, 1 (2016), 1–18.
- [69] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E Hassan, and Kenichi Matsumoto. 2018. The impact of automated parameter optimization on defect prediction models. *IEEE Transactions on Software Engineering* 45, 7 (2018), 683–711.
- [70] Ruben Van den Goorbergh, Maarten Van Smeden, Dirk Timmerman, and Ben Van Calster. 2022. The harm of class imbalance corrections for risk prediction models: illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association* 29, 9 (2022), 1525–1534.
- [71] Xiaohui Wan, Zheng Zheng, Fangyun Qin, and Xuhui Lu. 2024. Data complexity: a new perspective for analyzing the difficulty of defect prediction tasks. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–45.
- [72] Shuo Wang and Xin Yao. 2013. Using class imbalance learning for software defect prediction. *IEEE Transactions on Reliability* 62, 2 (2013), 434–443.
- [73] Michael Weiss and Paolo Tonella. 2022. Simple techniques work surprisingly well for neural network test prioritization and active learning (replicability study). In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 139–150.
- [74] Rongxin Wu, Hongyu Zhang, Sunghun Kim, and Shing-Chi Cheung. 2011. Relink: recovering links between bugs and changes. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of*

- software engineering. 15–25.
- [75] Ting-Fan Wu, Chih-Jen Lin, and Ruby C Weng. 2004. Probability estimates for multi-class classification by pairwise coupling. *Journal of Machine Learning Research* 5, Aug (2004), 975–1005.
- [76] Xianzong Wu, Xiaohong Li, Lili Quan, and Qiang Hu. 2025. UncertaintyZoo: A Unified Toolkit for Quantifying Predictive Uncertainty in Deep Learning Systems. *arXiv preprint arXiv:2512.06406* (2025).
- [77] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [78] Aidan ZH Yang, Sophia Kolak, Vincent Hellendoorn, Ruben Martins, and Claire Le Goues. 2025. Revisiting unnaturallness for automated program repair in the era of large language models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2561–2573.
- [79] Zezhi Ye, Chenghai Yu, and Zhilong Lu. 2025. Software Defect Prediction Model Based on AST and Deep Learning. *Scalable Computing: Practice and Experience* 26, 5 (2025), 2183–2195.
- [80] Xiao Yu, Jiqing Rao, Lei Liu, Guancheng Lin, Wenhua Hu, Jacky Wai Keung, Junwei Zhou, and Jianwen Xiang. 2024. Improving effort-aware defect prediction by directly learning to rank software modules. *Information and Software Technology* 165 (2024), 107250.
- [81] Bianca Zadrozny and Charles Elkan. 2002. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 694–699.
- [82] Yunhua Zhao, Kostadin Damevski, and Hui Chen. 2023. A systematic survey of just-in-time software defect prediction. *Comput. Surveys* 55, 10 (2023), 1–35.
- [83] Thomas Zimmermann, Nachiappan Nagappan, Harald Gall, Emanuel Giger, and Brendan Murphy. 2009. Cross-project defect prediction: a large scale experiment on data vs. domain vs. process. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*. 91–100.